# Four New Multi-Phase Hybrid Bracketing Algorithms for Numerical Root Finding

Abdelrahman Ellithy[1]
*Department of Artificial Intelligence, Faculty of Computers and Artificial Intelligence, Benha University, Benha, Egypt*

(Dated: November 27, 2025)

Finding roots of nonlinear equations requires balancing rapid convergence with guaranteed bracketing reliability. This work introduces four innovative high-performance hybrid algorithms — **Opt.BF, Opt.BFMS, Opt.TF, and Opt.TFMS** — These methods systematically combine robust interval contraction with accelerated interpolation and modified-secant corrections. Evaluated on a diverse **48-problem** benchmark, the leading method, Opt.BFMS, converges in only 2.79 iterations on average (23 wins in fewest iterations) using 3–4 function evaluations per iteration (NFE/iter) and 48–58 FLOPs/iter excluding function evaluations, yielding the lowest total estimated workload (approximately **401–650 FLOPs** per problem). This translates to an average CPU time of 10.48 microseconds — a 3.5x speedup over Bisection, 1.91x over Brent (SciPy), and **2.54x** over QIR — while winning 45.83% of problems by execution time and 47.9% by total NFE. These results establish the proposed hybrids, particularly Opt.BFMS, and Opt.TFMS as the new state-of-the-art in practical root-finding performance across polynomial, transcendental, and ill-conditioned equations.

**Keywords:** hybrid root-finding algorithms, root finding, Continuous optimization, convergence analysis, Derivative-free methods, Bracketing methods, modified secant method, Nonlinear equations, numerical analysis, numerical stability.

## I. INTRODUCTION

Finding roots is a basic problem in numerical analysis that is used in many fields, including physics, engineering, economics, and computer science. Convergence is guaranteed with traditional root bracketing methods like Bisection and Regula Falsi (False Position), but they often have slow convergence rates, and on the other hand, the open methods are faster but sometimes diverge. This trade-off between reliability and speed has led to the creation of hybrid methods that preserve the strength of bracketing methods while converging faster with multi-phase accelerated interpolation techniques. This strategy has shown considerable promise in earlier work [2, 3].

The field of root-finding algorithms has become more advanced beyond these traditional methods. Modern algorithms such as the widely used Brent's method [6] and Quadratic Interval Refinement (QIR) [7] are now considered benchmarks for high performance. Even though these techniques are successful, there is still a chance to create new algorithms that are either quicker or perform better. This paper addresses that gap by introducing four new hybrid algorithms and evaluating them not only against traditional baselines but also against these established, high-performance algorithms.

The main contributions of this paper are:

1. The design and implementation of **four novel hybrid algorithms** that blend reliable interval reduction utilize Bisection, Trisection and Regula Falsi with quasi-Newton steps.
2. A **detailed theoretical analysis** of the convergence order and error bounds for each proposed algorithm.
3. A **comprehensive public presentation evaluation** on a diverse benchmark of 48 test functions, augmented with notoriously difficult cases to trial robustness and numerical stability.
4. A **comparative analysis** against both traditional baselines and modern cutting-edge methods, using metrics that include function evaluations and performance in finite-precision arithmetic.
5. The empirical finding that our premier hybrid algorithm, **Opt.BFMS**, provides a superior balance of speed and reliability, demonstrably outperforming the established benchmarks on average.

This paper is organized as follows. Section II (page 2) reviews **related work** and contextualizes our contribution. Section III (page 3) details the **methodology** and mathematical foundations of the proposed algorithms. Section IV (page 11) provides a **Theoretical Analysis** of their convergence properties. Section V (page 13) describes the **experimental setup** and test functions. Section VI (page 21) presents the **results and analysis**. Section VII (page 30) discusses the implications. Section VIII (page 30) concludes the paper and outlines future research directions. Section IX (page 31) provides the **declarations**, including funding and competing interests statements.

## II. RELATED WORK AND CONTEXT

The concept of hybridizing root-finding methods to achieve both reliability and speed is well-established, dating back to the influential algorithm of the 1970s. Dekker's algorithm ( 1969 ) was a seminal contribution, combining the safety of bisection with the velocity of the secant method. It introduced the now-common strategy of falling back on a bisection step if the faster insertion measure fails to make sufficient progress. Brent's method acting [6] refined this idea by replacing the secant method with inverse quadratic interpolation, achieving a higher order of convergence while retaining the bisection safeguard, making it one of the most widely used and powerful root-finders today.

Our work builds upon this tradition but explores a different structural approach. As a conditional fallback, our algorithm applies a deterministic, multi-phase sequence inside each loop, whereas Dekker's and Brents' method uses bisection. Think about it this way: for example, the Opt.BFMS algorithmic rule systematically applies a bisection, a false position, and a modified secant step, leveraging the output of each to feed the next as input. Avoiding the complex acceptance criteria of classic hybrids that aim to maximize progress in every iteration.

More recently, Quadratic Interval Refinement (QIR) [7] has been proposed as a method that guarantees quadratic polynomial convergence for simple roots by fitting a quadratic model to three points to refine the root's location. While powerful, its performance can depend on the function's local behavior. The algorithms presented in this paper are differentiated by their reliance on simpler linear insertion and finite-difference steps, which, as our experiments show, can be more efficient on average across a wider variety of mapping types.

Moreover, this approach is different from some recent hybrid techniques included in our comparison, such as those given in Sabharwal(2019) [2] and Badr(2021) [3]. Such approaches usually employ a mixed one, including the conditional choice between the outputs: say, a bi-section or false position update within an iteration. Our advising algorithm is not so in any form as it follows deterministic **sequential multi-phase architecture**. For example, our Opt.BF does not really choose between bisection and falsi position, but, it is executing bisection and feeding the new output intervals as input to regula falsi. This fixed sequence procedure turns out to be a unique architectural choice to enforce procession with continuous interpolation-based acceleration.

## III. ALGORITHM DESIGN AND METHODOLOGY

This study introduces four hybrid root-finding algorithms that share a unified, deterministic multi-phase architecture. Unlike traditional hybrid methods (e.g., Brent, Dekker) that employ complex conditional logic to switch between methods, our approach executes a fixed sequence of distinct phases within each iteration. This design systematically leverages the reliability of interval contraction and the convergence speed of interpolation without the overhead of "choose-the-best" decision trees.

### A. General Multi-Phase Architecture

Each iteration $k$ of the proposed algorithms transforms the current bracketing interval $[a_k, b_k]$ into a refined interval $[a_{k+1}, b_{k+1}]$ through a cascade of up to three phases:

1. **Phase 1: Guaranteed Reduction.** A bracketing step (Bisection or Trisection) ensures the search space contracts by a fixed factor (1/2 or 1/3). This guarantees global convergence.
2. **Phase 2: Linear Interpolation.** A False Position (Regula Falsi) step is applied to the interval produced by Phase 1. This accelerates convergence when the function behaves linearly.
3. **Phase 3: Quadratic Acceleration (Optional).** A Modified Secant step estimates the root using a finite-difference approximation of the derivative. This step is accepted only if strictly robust safeguards are met.

### B. Computational Building Blocks

The algorithms are constructed from the following fundamental operators:

**Operator 1: Interval Halving (Bisection).** Given $[a, b]$, computes $x_b = (a + b)/2$. The sub-interval containing the root is retained.

**Operator 2: Interval Trisection.** Divides $[a, b]$ into three equal segments using $x_1 = a + (b - a)/3$ and $x_2 = b - (b - a)/3$. The algorithm evaluates $f(x_1)$ and $f(x_2)$ to identify the specific third containing the

root, offering a theoretical reduction factor of 3 per phase at the cost of two function evaluations.

**Operator 3: Regula Falsi (False Position).** Computes the linear interpolation root $x_{FP}$:

$$x_{FP} = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)} \tag{1}$$

To prevent division by zero, a machine-epsilon guard ($\epsilon = 10^{-15}$) is applied to the denominator.

**Operator 4: Modified Secant.** Approximates Newton's method using a finite difference step $\delta$. The candidate $x_S$ is computed from the current estimate $x_{FP}$:

$$x_S = x_{FP} - \frac{f(x_{FP}) \cdot \delta}{f(x_{FP} + \delta) - f(x_{FP})} \tag{2}$$

In this study, we employ an adaptive perturbation

$$\delta = 10^{-8} \times \max(1.0, |x_{FP}|) + 10^{-15}$$

to balance truncation and round-off errors, as recommended for robust numerical stability in derivative-free methods. This adaptive approach enhances generality across different root scales while maintaining performance in our benchmark. A candidate $x_S$ is accepted if and only if it resides strictly within the current bracket and yields a smaller function magnitude than $x_{curr}$.

### C. Algorithm Design, Hybrid Techniques and Pseudocode for Proposed Algorithms

The four algorithms are distinguished by their specific combination of these phases:

**1. Optimized Bisection-False Position (Opt.BF).** A two-phase sequence:

- *Step 1:* Execute Bisection.
- *Step 2:* Execute Regula Falsi on the result of Step 1.

**2. Optimized Bisection-False Position-Modified Secant (Opt.BFMS).** A three-phase sequence designed for maximum convergence speed:

- *Step 1:* Execute Bisection.
- *Step 2:* Execute Regula Falsi.
- *Step 3:* Execute Modified Secant on the result of Step 2, subject to acceptance criteria.

**3. Optimized Trisection-False Position (Opt.TF).** Replaces binary splitting with ternary splitting:

- *Step 1:* Execute Trisection.

- *Step 2:* Execute Regula Falsi on the chosen third.

**4. Optimized Trisection-False Position-Modified Secant (Opt.TFMS).** The most aggressive variant:

- *Step 1:* Execute Trisection.
- *Step 2:* Execute Regula Falsi.
- *Step 3:* Execute Modified Secant on the result of Step 2.

### *1. Algorithm 1: Optimized Bisection-False Position (Opt.BF)*

**Algorithm 1: Pseudocode for the Optimized Bisection-False Position (Opt.BF) algorithm. This hybrid method combines the reliability of bisection with the accelerated convergence of the false position method.**

**Input:** Function $f$, interval $[a, b]$, tolerance $tol$, max iterations $max_iter$
**Output:** Iteration count $n$, root approximation $x$, function value $f(x)$, final interval $[a, b]$
$f_a \leftarrow f(a)$, $f_b \leftarrow f(b)$, $\epsilon \leftarrow 10^{-15}$
*// Check if either bound is a root*
**if** $|f_a| \leq tol$ **then**
    **return** $1, a, f_a, a, b$
**end if**
**if** $|f_b| \leq tol$ **then**
    **return** $1, b, f_b, a, b$
**end if**
**if** $f_a \times f_b \geq 0$ **then**
    **return** $\text{None}, \text{None}, \text{None}, \text{None}, \text{None}$
**end if**
**for** $n \leftarrow 1$ **to** $max_iter$ **do**
    *// Bisection Step*
    $x_b \leftarrow 0.5 \times (a + b)$
    $f_{x_b} \leftarrow f(x_b)$
    **if** $|f_{x_b}| \leq tol$ **then**
        **return** $n, x_b, f_{x_b}, a, b$
    **end if**
    **if** $f_a \times f_{x_b} < 0$ **then**
        $b, f_b \leftarrow x_b, f_{x_b}$
    **else**
        $a, f_a \leftarrow x_b, f_{x_b}$
    **end if**
    *// 2. False Position Step with exception handling*
    $dx \leftarrow (a \times f_b) - (b \times f_a)$
    $dd \leftarrow f_b - f_a$
    **try:**

```
        x_FP ← dx/dd
    catch (division by zero, overflow, value error):
        x_FP ← dx/(dd + ϵ)
    f_{x_FP} ← f(x_FP)
    if |f_{x_FP}| ≤ tol then
        return n, x_FP, f_{x_FP}, a, b
    end if
    // Update interval based on false position
    if f_a × f_{x_FP} < 0 then
        b, f_b ← x_FP, f_{x_FP}
    else
        a, f_a ← x_FP, f_{x_FP}
    end if
end for
// Max iterations reached
final_x ← 0.5 × (a + b)
f_{final_x} ← f(final_x)
return max_iter, final_x, f_{final_x}, a, b
```

### 2. *Algorithm 2: Optimized Bisection-False Position with Modified Secant (Opt.BFMS)*

The modified secant estimate's acceptance criterion is what actually matters in the algorithm. First, it must lie within the current bracketing interval $[a, b]$, thereby satisfying the bracketing property. Second, it must provide a better approximation than the false position estimate, i.e., $|f(x_S)| < |f(x_{FP})|$. Only if both the conditions are satisfied, then $x_S$ is accepted as a valid secant approximation. Such a cautious approach assures that the algorithm never fails reliability just for the sake of speed.

**Algorithm 2: Pseudocode for the Hybrid Bisection-False Position-Modified Secant (Opt.BFMS) algorithm. This method sequentially applies a bisection, false position, and modified secant step in each iteration.**

```
Input: Function f, interval [a, b], tolerance tol, max iterations max_iter
Output: Iteration count n, root approximation x, function value f(x), final interval [a, b]
f_a ← f(a), f_b ← f(b), n ← 0, ϵ ← 10^{-15}
// Check if either bound is a root
if |f_a| ≤ tol then
    return 1, a, f_a, a, b
end if
if |f_b| ≤ tol then
    return 1, b, f_b, a, b
end if
if f_a × f_b ≥ 0 then
    return None, None, None, None, None
end if
while n < max_iter do
    n ← n + 1
```

```
    // 1. Bisection Step to narrow the interval
    x_B ← (a + b) × 0.5
    f_{x_B} ← f(x_B)
    if f_a × f_{x_B} < 0 then
        b ← x_B, f_b ← f_{x_B}
    else
        a ← x_B, f_a ← f_{x_B}
    end if
    // 2. False Position Step with exception handling
    dx ← (a × f_b) − (b × f_a)
    dd ← f_b − f_a
    try:
       x_FP ← dx/dd
    catch (division by zero, overflow, value error):
       x_FP ← dx/(dd + ϵ)
    f_{x_FP} ← f(x_FP)
    if |f_{x_FP}| ≤ tol then
        return n, x_FP, f_{x_FP}, a, b
    end if
    // Update interval based on false position
    if f_a × f_{x_FP} < 0 then
        b, f_b ← x_FP, f_{x_FP}
    else
        a, f_a ← x_FP, f_{x_FP}
    end if
    // 3. Modified Secant Step with exception handling
    δ ← 10^{−8} max(1, |x_FP|) + 10^{−15}
    try:
       x_S ← x_FP − ((δ × f_{x_FP})/(f(x_FP + δ) − f_{x_FP}) )
    catch (division by zero, overflow, value error):
       x_S ← x_FP − ((δ × f_{x_FP})/((f(x_FP + δ) − f_{x_FP}) + ϵ) )
    if a < x_S < b then
        f_{x_S} ← f(x_S)
        if |f_{x_S}| < |f_{x_FP}| then
            if f_a × f_{x_S} < 0 then
                b ← x_S, f_b ← f_{x_S}
            else
                a ← x_S, f_a ← f_{x_S}
            end if
            if |f_{x_S}| ≤ tol then
                return n, x_S, f_{x_S}, a, b
            end if
        end if
    end if
end while
```

*// Max iterations reached, return the midpoint of the final interval*
$final_x \leftarrow 0.5 \times (a + b)$
$f_{final_x} \leftarrow f(final_x)$
**return** $max_iter, final_x, f_{final_x}, a, b$

### 3. Algorithm 3: Optimized Trisection-False Position (Opt.TF)

**Algorithm 3: Pseudocode for the Optimized Trisection-False Position (Opt.TF) algorithm. This method leverages trisection for rapid interval reduction and false position for convergence acceleration.**

**Input:** Function $f$, interval $[a, b]$, tolerance $tol$, max iterations $max_iter$
**Output:** Root approximation $x$, function value $f(x)$, final interval $[a, b]$
$f_a \leftarrow f(a)$, $f_b \leftarrow f(b)$, $\epsilon \leftarrow 10^{-15}$
*// Check if either bound is a root*
**if** $|f_a| \leq tol$ **then**
    **return** $1, a, f_a, a, b$
**end if**
**if** $|f_b| \leq tol$ **then**
    **return** $1, b, f_b, a, b$
**end if**
**for** $n \leftarrow 1$ **to** $max_iter$ **do**
    *// 1. Trisection Step*
    $diff \leftarrow b - a$
    $x_1 \leftarrow a + diff/3$
    $x_2 \leftarrow b - diff/3$
    $f_{x_1} \leftarrow f(x_1)$, $f_{x_2} \leftarrow f(x_2)$
    **if** $|f_{x_1}| \leq tol$ **then**
        **return** $n, x_1, f_{x_1}, a, b$
    **end if**
    **if** $|f_{x_2}| \leq tol$ **then**
        **return** $n, x_2, f_{x_2}, a, b$
    **end if**
    *// Update interval based on trisection*
    **if** $f_a \times f_{x_1} < 0$ **then**
        $b, f_b \leftarrow x_1, f_{x_1}$
    **else if** $f_{x_1} \times f_{x_2} < 0$ **then**
        $a, b, f_a, f_b \leftarrow x_1, x_2, f_{x_1}, f_{x_2}$
    **else**
        $a, f_a \leftarrow x_2, f_{x_2}$
    **end if**
    *// 2. False Position Step with exception handling*
    $dx \leftarrow (a \times f_b) - (b \times f_a)$
    $dd \leftarrow f_b - f_a$
    **try:**

```
        x_FP ← dx/dd
    catch (division by zero, overflow, value error):
        x_FP ← dx/(dd + ϵ)
    f_{x_FP} ← f(x_FP)
    if |f_{x_FP}| ≤ tol then
        return n, x_FP, f_{x_FP}, a, b
    end if
    // Update interval based on false position
    if f_a × f_{x_FP} < 0 then
        b, f_b ← x_FP, f_{x_FP}
    else
        a, f_a ← x_FP, f_{x_FP}
    end if
end for
// Max iterations reached
final_x ← 0.5 × (a + b)
f_{final_x} ← f(final_x)
return max_iter, final_x, f_{final_x}, a, b
```

*4. Algorithm 4: Optimized Trisection-False Position with Modified Secant (Opt.TFMS)*

**Algorithm 4: Pseudocode for the Optimized Trisection-False Position with Modified Secant (Opt.TFMS) algorithm. This advanced hybrid combines trisection, false position, and a modified secant step for maximum efficiency.**

```
Input: Function f, interval [a, b], tolerance tol, max iterations max_iter
Output: Root approximation x, function value f(x), final interval [a, b]
f_a ← f(a), f_b ← f(b), ϵ ← 10^{-15}
// Check if either bound is a root
if |f_a| ≤ tol then
    return 1, a, f_a, a, b
end if
if |f_b| ≤ tol then
    return 1, b, f_b, a, b
end if
if f_a × f_b ≥ 0 then
    return None, None, None, None, None
end if
for n ← 1 to max_iter do
    // 1. Trisection Step
    diff ← b − a
    third ← (diff/3)
    x_1 ← a + third, x_2 ← b − third
    f_{x_1} ← f(x_1), f_{x_2} ← f(x_2)
    if |f_{x_1}| ≤ tol then
```

```
        return n, x_1, f_{x_1}, a, b
    end if
    if |f_{x_2}| ≤ tol then
        return n, x_2, f_{x_2}, a, b
    end if
    // Update interval based on trisection results
    if f_a × f_{x_1} < 0 then
        b, f_b ← x_1, f_{x_1}
    else if f_{x_1} × f_{x_2} < 0 then
        a, b, f_a, f_b ← x_1, x_2, f_{x_1}, f_{x_2}
    else
        a, f_a ← x_2, f_{x_2}
    end if
    // 2. False Position Step with exception handling
    dx ← (a × f_b) − (b × f_a)
    dd ← f_b − f_a
    try:
      x_FP ← dx/dd
    catch (division by zero, overflow, value error):
      x_FP ← dx/(dd + ϵ)
    f_{x_FP} ← f(x_FP)
    if |f_{x_FP}| ≤ tol then
        return n, x_FP, f_{x_FP}, a, b
    end if
    // Update interval based on false position
    if f_a × f_{x_FP} < 0 then
        b, f_b ← x_FP, f_{x_FP}
    else
        a, f_a ← x_FP, f_{x_FP}
    end if
    // 3. Modified Secant Step with exception handling
    δ ← 10^{-8} max(1, |x_FP|) + 10^{-15}
    try:
      x_S ← x_FP − δ × f_{x_FP}/(f(x_FP + δ) − f_{x_FP})
    catch (division by zero, overflow, value error):
      x_S ← x_FP − δ × f_{x_FP}/((f(x_FP + δ) − f_{x_FP}) + ϵ)
    if a < x_S < b then
        f_{x_S} ← f(x_S)
        if |f_{x_S}| < |f_{x_FP}| then
            if f_a × f_{x_S} < 0 then
                b, f_b ← x_S, f_{x_S}
            else
                a, f_a ← x_S, f_{x_S}
            end if
            if |f_{x_S}| ≤ tol then
```

```
            return n, x_S, f_{x_S}, a, b
        end if
    end if
  end if
end for
// Max iterations reached
final_x ← 0.5 × (a + b)
f_{final_x} ← f(final_x)
return max_iter, final_x, f_{final_x}, a, b
```

## IV. THEORETICAL ANALYSIS

A primary contribution of this work is a rigorous theoretical analysis of the convergence properties of the proposed hybrid algorithms. While the Intermediate Value Theorem guarantees their eventual convergence, a specific analysis of the multi-phase structure is required to establish the order of convergence and formally characterize the performance bounds.

### A. Order of Convergence

Let $\alpha$ be a simple root of $f(x)$ with $f'(\alpha) \neq 0$ and $f''$ continuous near $\alpha$. Let $e_n = x_n - \alpha$ denote the error at iteration $n$.

#### *1. Two-Phase Variants (Opt.BF and Opt.TF)*

These algorithms apply classical Regula Falsi (False Position) as the acceleration phase. It is well established that unmodified Regula Falsi converges linearly in general and can stagnate when one bracketing endpoint becomes fixed [1]. However, the preceding bisection or trisection phase continuously refreshes both endpoints, eliminating stagnation. While the asymptotic convergence order remains linear, the effective linear convergence constant is dramatically improved, yielding substantially faster practical convergence than standalone False Position.

#### *2. Three-Phase Variants (Opt.BFMS and Opt.TFMS)*

When the modified-secant step is accepted, the update approximates Newton's method with a forward-difference derivative. The error relation for Newton's method is given by $e_{n+1} = Ce_n^2 + O(e_n^3)$. Our finite difference approximation of the derivative introduces a perturbation, as shown by its Taylor expansion:

$$\frac{f(x_n + \delta) - f(x_n)}{\delta} = f'(x_n) + \frac{\delta}{2} f''(x_n) + O(\delta^2)$$

Using the adaptive choice $\delta = 10^{-8}\max(1, |x_n|) + 10^{-15} \approx \sqrt{\epsilon_{\text{mach}}} \cdot |x_n|$, the dominant error term is quadratic, but a linear perturbation proportional to $\delta$ persists. Thus, local convergence is **faster than linear but slower than full quadratic** — consistent with perturbed Newton methods [5].

*3. Theorem (Local convergence order).*

Let $\alpha$ be a simple root of $f \in C^2$ near $\alpha$ with $f'(\alpha) \neq 0$. If the modified-secant candidate remains inside the current bracket and satisfies $|f(x_S)| < |f(x_n)|$ in at least a fixed positive fraction $\gamma > 0$ of the iterations sufficiently close to the root, then the observed asymptotic convergence order $p$ of Opt.BFMS and Opt.TFMS satisfies

$$1 < p < 2.$$

Empirical acceptance rates $\gamma \gtrsim 0.70$ on well-conditioned problems (Section VI) typically produce effective orders in the range $1.6 \leq p \leq 1.9$.

**Lower Bound (Graceful Degradation):** When the modified-secant step is rejected, the algorithm falls back to the two-phase variant with refreshed brackets. This avoids endpoint stagnation and converges at least as fast as False Position on a continuously updated interval — strictly to classical Regula Falsi in practice, though still asymptotically linear.

Thus, the three-phase algorithms guarantee convergence no slower than improved linear, while opportunistically achieving superlinear acceleration typically 1.6 ¡= p ¡= 1.9 when conditions permit.

### B. Finite-Precision Safeguards

The primary source of instability in these methods is the denominator of the interpolation formulas. For both the false position and modified secant steps, these take the form $f(y) - f(z)$ for two points $y$ and $z$. In standard floating-point arithmetic (IEEE 754 double precision), when $y$ and $z$ are very close to each other, their function values $f(y)$ and $f(z)$ will also be very close, so when perform The subtraction of these two nearly equal numbers, it can lead to **catastrophic cancellation**, a phenomenon where a large portion of the significant digits are lost, resulting in a denominator dominated by relative error. Our pseudocode's inclusion of a small machine epsilon ($\epsilon = 10^{-15}$) is a pragmatic heuristic to prevent division by zero.

Furthermore, the choice of the perturbation parameter $\delta$ in the modified secant method is critical and represents a fundamental trade-off between both truncation and round-off errors.

- **Truncation Error:** If $\delta$ is too large, the finite difference $\frac{f(x+\delta)-f(x)}{\delta}$ is a poor approximation of the derivative $f'(x)$, and the method's convergence slows. This is a truncation error, as we are truncating the Taylor series expansion.
- **Round-off Error:** If $\delta$ is too small, it becomes comparable to the machine precision. The term $f(x+\delta)$ will be very close to $f(x)$, and their subtraction will suffer from the catastrophic cancellation described above. This is a round-off error.

An optimal choice for $\delta$ balances these two error sources, and we use the common heuristic $\delta \approx 10^{-8} \cdot \max(1.0, |x|) + 10^{-15}$, where the scaling approximates $\sqrt{\epsilon_{\mathbf{mach}}} \cdot |x|$ and $\epsilon_{\mathbf{mach}}$ is the machine epsilon (approx. $10^{-16}$ for double precision). This **adaptive choice** provides robust performance and stability across varying root scales in our benchmark. A candidate $x_S$ is accepted if and only if it resides strictly within the current bracket and yields a smaller function magnitude than $x_{curr}$. The impact of these finite-precision effects is empirically evaluated in Section V.

## V. EXPERIMENTAL SETUP

We designed a rigorous experimental framework to evaluate the proposed algorithms against established baselines (Bisection, False Position) and state-of-the-art methods (Brent's Method, QIR).

### A. Implementation and Compilation Pipeline

To ensure a fair assessment of computational efficiency, all algorithms were implemented using the SymPy library for symbolic definition but were **pre-compiled** for execution.

**Symbolic Function Definition:** The functions are defined using SymPy symbols, allowing for exact mathematical characterization. This ensures that the test functions are represented without initial numerical approximation errors.

**Compilation for Performance (sp.lambdify):** A critical feature of our experimental setup is the use of `sp.lambdify` with the default **NumPy** backend. While SymPy handles the symbolic definition, the **actual execution during benchmarking** is performed by **vectorized NumPy** functions generated from the symbolic expressions. This **eliminates virtually all interpretation overhead** of the symbolic engine itself, so the reported CPU times predominantly reflect the efficiency of the numerical algorithm rather than SymPy's expression evaluation.

**Comprehensive Error Handling:** The implementation features extensive error surveillances for rare cases like division by zero, overflow conditions, and convergence failures.

**Adaptive Parameters:** The modified secant step uses an adaptive $\delta$ computed per iteration for enhanced numerical robustness.

**Single-Loop Strategy:** We utilized a high-volume single-loop strategy. Each algorithm was executed $N = 10,000$ times per problem.

**Garbage Collection Control:** To prevent Python's automatic memory management from skewing microsecond-level measurements, the Garbage Collector (GC) was explicitly disabled immediately prior to the critical execution block and re-enabled immediately after.

We used a strict timing protocol to obtain appropriate statistics while minimizing the effect of noise from the system (such as OS interrupts or memory management).

### B. Hardware and Software Environment Details

All experiments were conducted on GitHub Actions standard hosted runners (Linux Ubuntu 24.04.3 LTS, Kernel 6.11.0-1018-azure). It is noted that the underlying hardware for these runners is dynamically allocated and may vary between sessions; however, the results reported here were obtained on a virtualized instance backed by an AMD EPYC 7763 Processor (2 vCPUs allocated) and 7 GB RAM. The software environment included Python 3.11.7, NumPy 2.3.5, SciPy 1.16.3, and SymPy 1.14.0. Timing used **time.perf_counter_ns()** with 100-iteration warm-up and 10,000 timed repetitions per problem. Convergence was declared when both

$$|b - a| < 10^{-14} \quad \text{and} \quad |f(x)| < 10^{-14}$$

were satisfied, or after 10,000 iterations. All solvers used identical stopping criteria and function objects. Complete source code, benchmark suite, and raw results are available as detailed in Section IX D.

### C. Performance Evaluation and metrics

To assure a comprehensive and fair comparison, all algorithms were evaluated on a set of theoretical, implementation-independent, and practical-real-world metrics, measurements were performed with strict statistical rigor: each algorithm was executed **10000 times on every problem**, using Python's high-resolution `time.perf_counter_ns()` timer, with results reported in **microseconds (µs)**. All benchmarks were conducted on the same isolated machine to minimize system noise.

The following metrics were recorded and analyzed:

**Number of Iterations**: The number of main algorithmic loops required for convergence. Lower values indicate faster theoretical convergence, but must be interpreted alongside per-iteration cost.

**Function Evaluations per Iteration (NFE/iter)**: Typical number of function calls performed in each iteration, this is a crucial measure of having efficiency unrelated to implementation

**Estimated FLOPs per Iteration**: FLOP estimation per iteration through static code analysis (arithmetic operations, conditionals, and accesses to memory). The ranges reflect typical execution paths (e.g., 48-58 for Opt.BFMS, 85-100 for Brent).

**Total Estimated FLOPs per Problem**: Computed as Avg Iterations × (NFE/iter range) × (FLOPs/iter range). This metric captures the complete theoretical computational workload and explains observed timing differences even when iteration counts are similar.

**CPU Time ($\mu$s)**: Average wall-clock execution time per problem (across 10000 repetitions). This is the resource counting last practical performance measure reflecting extents of real-world importance in resource-constrained or high-frequency numerical applications.

**Wins (Speed and Convergence)**: Number of problems (out of 48) where an algorithm achieved the lowest CPU time or fewest iterations. These counts serve as robust, non-parametric indicators of dominance across diverse function classes.

**Accuracy Metrics**: - Absolute function value at the computed root $|f(x)| \leq 10^{-14}$ (enforced tolerance). - Final bracketing interval length, confirming all methods achieve machine-precision-level accuracy when convergence is declared.

By combining hardware-independent theoretical metrics (iterations, NFE, total FLOPs) with rigorous empirical timing and robustness analysis, this evaluation framework provides a complete, transparent, and scientifically sound basis for comparing root-finding algorithms — far beyond what is possible with iteration count or function evaluations alone.

This methodology ensures that the performance differences reported in Section VI are attributable to the algorithmic logic itself rather than implementation artifacts.

### D. Benchmark Test Suite

The algorithms were stress-tested on a diverse suite of **48 benchmark functions**. This dataset was constructed to cover a wide spectrum of mathematical behaviors, ranging from simple continuous functions to pathological cases with high-frequency oscillations, singularities, and high-degree polynomials (up to degree 25).

The test functions are categorized by their dominant mathematical characteristics:

**Transcendental Functions:**

- $f_1(x) = xe^x - 7$: A transcendental function combining exponential and linear terms, interval $[1, 2]$
- $f_8(x) = e^x - 3x - 2$: A mixed exponential-linear function, interval $[2, 3]$
- $f_{15}(x) = e^x - 3x^2$: Exponential-quadratic combination, interval $[0, 1]$
- $f_{18}(x) = x^2 - e^x - 3x + 2$: Quadratic-exponential-linear combination, interval $[0, 1]$
- $f_{20}(x) = e^{x^2} - e^{\sqrt{2}x}$: Complex exponential function, interval $[0.5, 1.5]$
- $f_{29}(x) = e^{-x} - \cos(3x) - 0.5$: Mixed exponential-trigonometric function, interval $[0, 1]$
- $f_{39}(x) = e^{x^2} - e^{1.2x}$: Exponential function with different exponents, interval $[0, 2]$
- **Transcendental growth model (Financial Volatility Proxy)**

$$f_{46}(x) = xe^{x/2} - 1.5, \quad I = [0.0, 1.0]$$

**High-Degree Polynomial Functions:**

- $f_2(x) = x^3 - x - 1$: A cubic polynomial with multiple roots, interval $[1, 2]$
- $f_{12}(x) = x^{10} - 1$: A tenth-degree polynomial function, interval $[0, 1.3]$
- $f_{14}(x) = x^3 - 2x - 5$: Another cubic polynomial, interval $[2, 3]$
- $f_{19}(x) = (x - 1)^3 + 4(x - 1)^2 - 10$: A cubic polynomial, interval $[0, 3]$
- $f_{21}(x) = (x^2 - 4)(x + 1.5)(x - 0.5)$: A quartic polynomial, interval $[0, 2]$
- $f_{22}(x) = x^3 - 3x^2 - 4x + 13$: A cubic polynomial, interval $[-3, -2]$
- $f_{26}(x) = x^5 - 3x^4 + 25$: A fifth-degree polynomial, interval $[-3, -1]$

- $f_{27}(x) = x^4 - 2x^2 - 4$: A quartic polynomial, interval $[1, 3]$
- $f_{32}(x) = x^4 + 2x^2 - x - 1$: A quartic polynomial, interval $[-0.5, 0]$
- $f_{33}(x) = x^4 - 10x^3 + 35x^2 - 50x + 24$: A quartic polynomial, interval $[0, 1.5]$
- $f_{35}(x) = x^{25} - 1$: A twenty-fifth-degree polynomial (ill-conditioned), interval $[0, 2]$
- $f_{38}(x) = x^4 + 2x^3 - 13x^2 - 14x + 24$: A quartic polynomial, interval $[-3, 1]$
- $f_{40}(x) = x^5 - 3x^4 + 2x^3 - x + 0.1$: A fifth-degree polynomial with small coefficients, interval $[-1, 3]$
- $f_{41}(x) = (x + 0.3)^7 - 0.01$: A seventh-degree polynomial with small perturbation, interval $[-2, 1]$
- $f_{42}(x) = x^6 - 8x^5 + 24x^4 - 32x^3 + 16x^2$: A sixth-degree polynomial, interval $[0, 4]$
- **Van der Waals equation of state for $CO_2$ (Thermodynamics)**

$$f_{45}(x) = \left(10 + \frac{3.592}{x^2}\right)(x - 0.04267) - 24.618, \quad I = [2.0, 3.0]$$

**Quadratic Functions:**

- $f_3(x) = x^2 - x - 2$: A standard quadratic equation, interval $[1, 4]$
- $f_5(x) = x^2 - 10$: A simple quadratic with irrational roots, interval $[3, 4]$
- $f_{13}(x) = x^2 + 2x - 7$: A quadratic with different coefficients, interval $[1, 3]$
- $f_{23}(x) = -0.9x^2 + 1.7x + 2.5$: A quadratic with negative leading coefficient, interval $[2.8, 3.0]$
- $f_{24}(x) = 1 - 0.61x$: A linear function, interval $[1.5, 2.0]$

**Trigonometric Functions:**

- $f_4(x) = x - \cos(x)$: A mixed linear-trigonometric function, interval $[0, 1]$
- $f_6(x) = \sin(x) - x^2$: A trigonometric-quadratic combination, interval $[0.5, 1]$
- $f_{10}(x) = x\sin(x) - 1$: A product of linear and trigonometric terms, interval $[0, 2]$
- $f_{11}(x) = x\cos(x) + 1$: A product of linear and trigonometric terms, interval $[-2, 4]$
- $f_{16}(x) = \sin(10x) - 0.5x$: A high-frequency trigonometric function, interval $[0.1, 0.4]$
- $f_{17}(x) = x - 0.8\sin(x) - 1.2$: A mixed linear-trigonometric function, interval $[1, 3]$
- $f_{25}(x) = x^2|\sin(x)| - 4.1$: A trigonometric function with absolute value, interval $[0, 4]$
- $f_{28}(x) = x - 0.5\sin(x) - 1$: A mixed linear-trigonometric function, interval $[0, 3]$
- $f_{34}(x) = 4\sin(x) - x + 1$: A trigonometric-linear combination, interval $[-1, 0]$
- $f_{37}(x) = \sin(20x) - 0.3x$: A very high-frequency trigonometric function, interval $[0.05, 0.25]$

- **Non-linear beam deflection (Structural Engineering)**

$$f_{47}(x) = \cos(x) + (1 - \cos(x))^2 - 0.05x^2, \quad I = [3.0, 6.0]$$

- **Kepler's equation for high eccentricity (Orbital Mechanics)**

$$f_{44}(x) = x - 0.99\sin(x) - 2.0, \quad I = [2.0, 3.0]$$

**Logarithmic and Mixed Functions:**

- $f_7(x) = x + \ln(x)$: A mixed linear-logarithmic function, interval $[0.1, 1]$

- $f_9(x) = x^2 + e^{x/2} - 5$: A quadratic-exponential combination, interval $[1, 2]$

- **Colebrook-White friction factor (Fluid Mechanics)**

$$f_{43}(x) = -2\log_{10}\left(2.7 \times 10^{-5} + \frac{2.51}{10^7\sqrt{x}}\right) - \frac{1}{\sqrt{x}}, \quad I = [0.008, 0.03]$$

**Ill-Conditioned and Challenging Functions:**

- $f_{30}(x) = (x-1)(x-2)\cdots(x-20)$: A twentieth-degree polynomial with integer roots, interval $[0, 1.5]$

- $f_{31}(x) = (x-1)(x-2)\cdots(x-20)$: A twentieth-degree polynomial with integer roots in a different interval, $[19, 21]$

- $f_{36}(x) = (x-1.8)^6(x-1.81)$: An ill-conditioned polynomial with close roots, interval $[0, 2]$

- **Chemical equilibrium reaction extent**

$$f_{48}(x) = 0.05 - \frac{x^3}{(1-x)(0.8-2x)^2}, \quad I = [0.01, 0.3]$$

Each function was evaluated over carefully selected intervals that bracket the root, with initial intervals chosen to ensure the Intermediate Value Theorem conditions are satisfied.

### E. Algorithm Code Legend

**TABLE I: Algorithm code legend used throughout all tables. Each code corresponds to a specific root-finding algorithm, as defined in the methodology.**

| Code | Algorithm Name |
|---|---|
| Bisection | Bisection |
| False Position | Regula Falsi (False Position) |
| Trisection | Trisection |
| HybridBlendTF | Badr-2021 - Hybrid Blend Trisection False Position |
| HybridBlendBF | Sabharwal 2019 - Blended Root Finding Algorithm Outperforms Bisection and Regula Falsi |
| Brent (SciPy) | Brent (SciPy implementation) |
| Opt.BF | Optimized Hybrid Bisection False Position |
| Opt.BFMS | Optimized Hybrid Bisection False Position Modified Secant |
| Opt.TF | Optimized Hybrid Trisection False Position |
| Opt.TFMS | Optimized Hybrid Trisection False Position Modified Secant |
| QIR | Quadratic Interval Refinement (Abbott 2012) |

### Case Study 1: Cubic Polynomial Test Case (Problem 19)

To evaluate performance on a standard polynomial function, we analyze Problem 19 from the benchmark set. This case study demonstrates the algorithms' efficiency on a common class of problems found in numerical analysis. The test function is the cubic polynomial:

$$f(x) = (x-1)^3 + 4(x-1)^2 - 10,$$

on the interval $[0, 3]$. Table II summarizes the results for all algorithms on this problem.

**TABLE II: Performance on the cubic polynomial problem $(x-1)^3 + 4(x-1)^2 - 10$ over $[0, 3]$. Metrics reported are average CPU time (in microseconds) over 10,000 runs.**

| Algorithm | Avg CPU Time (µs) | Rel. CPU Speed | Iterations | Rel. Iterations |
|---|---|---|---|---|
| **Opt.BFMS** | **5.5338** | **1.00**x (fastest) | **3.0** | **1.00**x (fewest) |
| Opt.TFMS | 6.6823 | 1.21x | 3.0 | 1.00x |
| Opt.BF | 6.9964 | 1.26x | 9.0 | 3.00x |
| Opt.TF | 7.9149 | 1.43x | 7.0 | 2.33x |
| False Position | 10.4817 | 1.89x | 28.0 | 9.33x |
| QIR | 14.1195 | 2.55x | 5.0 | 1.67x |
| HybridBlendBF | 15.8010 | 2.86x | 11.0 | 3.67x |
| HybridBlendTF | 16.4268 | 2.97x | 9.0 | 3.00x |
| Bisection | 17.8063 | 3.22x | 51.0 | 17.00x |
| Brent (SciPy) | 18.4091 | 3.33x | 11.0 | 3.67x |
| Trisection | 24.0371 | 4.34x | 32.0 | 10.67x |

The results again highlight the efficiency of the optimized hybrid methods, with Opt.BFMS leading the performance on this cubic polynomial, followed closely by the other proposed hybrid variants.

### Case Study 2: Kepler's Equation (Problem 44)

To further demonstrate algorithm behaviour on a representative transcendental test from Orbital Mechanics, we consider Problem 44 from the benchmark set:

$$f(x) = x - 0.99\sin(x) - 2.0,$$

on the interval $[2.0, 3.0]$. Table III summarizes the average CPU time (in microseconds) for each algorithm on this problem and their relative performance compared to the fastest method.

**TABLE III: Performance on $f(x) = x - 0.99\sin(x) - 2.0$ over the interval $[2.0, 3.0]$. Metrics reported are average CPU time (in $\mu s$) over 10,000 runs.**

| Algorithm | Avg CPU Time (µs) | Rel. CPU Speed | Iterations | Rel. Iterations |
|---|---|---|---|---|
| **Opt.BFMS** | **11.7251** | **1.00x** (fastest) | **2.0** | **1.00x** (fewest) |
| **Opt.TFMS** | 14.5426 | 1.24x | **2.0** | **1.00x** |
| False Position | 15.4351 | 1.32x | 11.0 | 5.50x |
| Opt.BF | 18.4329 | 1.57x | 7.0 | 3.50x |
| Opt.TF | 19.0057 | 1.62x | 5.0 | 2.50x |
| Brent (SciPy) | 20.2144 | 1.72x | 8.0 | 4.00x |
| QIR | 30.6418 | 2.61x | 7.0 | 3.50x |
| HybridBlendTF | 32.4390 | 2.77x | 6.0 | 3.00x |
| HybridBlendBF | 34.3589 | 2.93x | 8.0 | 4.00x |
| Bisection | 50.9900 | 4.35x | 47.0 | 23.50x |
| Trisection | 60.9949 | 5.20x | 28.0 | 14.00x |

The results for Problem 44 indicate that the optimized BFMS variant remains the most efficient solver for this high-eccentricity Kepler problem. It significantly outperforms the other methods.

### Case Study 3: Hardest Benchmark Problem (Problem 30, the classic Wilkinson's polynomial)

To demonstrate practical applicability on computationally demanding tasks, we analyze Problem 30, the classic Wilkinson's polynomial, the most challenging function in our test suite. This problem is a twentieth-degree polynomial with known integer roots:

$$f(x) = \prod_{k=1}^{20} (x - k)$$

on the interval $[0, 1.5]$. Table IV summarizes the results for this worst-case scenario.

**TABLE IV: Performance on the hardest benchmark problem, a twentieth-degree polynomial, over $[0, 1.5]$. Metrics reported are average CPU time (in microseconds) over 10,000 runs.**

| Algorithm | Avg CPU Time (µs) | Rel. CPU Speed | Iterations | Rel. Iterations |
|---|---|---|---|---|
| **Trisection** | **3.1677** | **1.00x** (fastest) | 1.0 | 1.0x (fewest) |
| **Opt.TFMS** | 3.3805 | 1.07x | **1.0** | **1.0x** (fewest) |
| Opt.TF | 3.4113 | 1.08x | 1.0 | 1.0x |
| HybridBlendTF | 3.9321 | 1.24x | 1.0 | 1.0x |
| Opt.BFMS | 16.1270 | 5.09x | 5.0 | 5.0x |
| Opt.BF | 19.4012 | 6.13x | 11.0 | 11.0x |
| Brent (SciPy) | 28.4101 | 8.97x | 14.0 | 14.0x |
| QIR | 30.3394 | 9.58x | 9.0 | 9.0x |
| Bisection | 42.8779 | 13.54x | 52.0 | 52.0x |
| HybridBlendBF | 45.7108 | 14.44x | 14.0 | 14.0x |
| False Position | 7947.57 | 2510x | 10000.0 | 10000x |

On this exceptionally difficult problem, the simple Trisection method finds the root most efficiently. However, the proposed trisection-based hybrids, Opt.TF and Opt.TFMS, perform nearly as well, ranking second and third. This demonstrates their value as robust alternatives.

### Case Study 4: Chemical Equilibrium (Problem 48)

To demonstrate performance on ill-conditioned rational functions, we analyze Problem 48 from our test suite. This function describes the extent of reaction in chemical equilibrium and presents a challenge due to the singularity near the boundary.

$$f_{48}(x) = 0.05 - \frac{x^3}{(1-x)(0.8-2x)^2}$$

on the interval $[0.01, 0.3]$. Table V summarizes the results for all algorithms on this problem.

**TABLE V: Performance on the chemical equilibrium function $f_{48}(x)$ over $[0.01, 0.3]$. Metrics reported are average CPU time (in microseconds) over 10,000 runs.**

| Algorithm | Avg CPU Time (µs) | Rel. CPU Speed | Iterations | Rel. Iterations |
|---|---|---|---|---|
| **Opt.BFMS** | **6.1473** | **1.00x** (fastest) | 4.0 | 1.33x |
| Opt.TFMS | 6.8024 | 1.11x | **3.0** | **1.00x** (fewest) |
| Opt.BF | 6.8302 | 1.11x | 9.0 | 3.00x |
| Opt.TF | 7.0236 | 1.14x | 6.0 | 2.00x |
| HybridBlendBF | 12.7775 | 2.08x | 9.0 | 3.00x |
| HybridBlendTF | 14.4400 | 2.35x | 8.0 | 2.67x |
| Bisection | 15.3825 | 2.50x | 44.0 | 14.67x |
| QIR | 16.6203 | 2.70x | 6.0 | 2.00x |
| Trisection | 19.9603 | 3.25x | 27.0 | 9.00x |
| Brent (SciPy) | 21.5799 | 3.51x | 13.0 | 4.33x |
| False Position | 65.9793 | 10.73x | 187.0 | 62.33x |

On this challenging rational function, the proposed Opt.BFMS algorithm demonstrates great performance, achieving the fastest convergence.

## VI. RESULTS AND ANALYSIS

This section presents a comprehensive analysis of the performance data, including detailed comparisons across all algorithms and test functions, statistical significance of the results, and practical implications for numerical computing applications.

### A. Overall Performance Analysis

The proposed multi-phase hybrid algorithms achieve major gains in both measured wall-clock time and theoretical efficiency. Table VI presents the comprehensive performance metrics for all twelve algorithms, including the four proposed hybrid methods, five traditional/baseline methods, and three state-of-the-art methods for comparison.

**TABLE VI: Comprehensive comparison of all root-finding algorithms across 48 diverse benchmark problems. Reported metrics are the average CPU time (in microseconds), median CPU time, standard deviation, and number of wins (problems where the algorithm achieved the fastest time). Algorithm codes are defined in Table I.**

| Algorithm | Avg CPU Time (us) | Min CPU Time (us) | Max CPU Time (us) | Median CPU Time (us) | Std Dev (us) | Wins |
|---|---|---|---|---|---|---|
| Bisection | 36.92177639 | 0.501 | 512.103 | 41.307 | 35.46476754 | 2 |
| False Position | 365.4653505 | 0.48 | 14475.096 | 16.721 | 1275.855078 | 8 |
| Trisection | 274.6785102 | 0.801 | 19102.918 | 54.201 | 1551.503378 | 3 |
| HybridBlendTF | 25.79302505 | 0.851 | 343.25 | 16.101 | 27.26079057 | 0 |
| HybridBlendBF | 25.50264367 | 0.621 | 512.274 | 16.381 | 27.55572859 | 0 |
| QIR | 26.61931221 | 0.781 | 642.026 | 15.989 | 31.79628482 | 2 |
| Brent (SciPy) | 19.97811586 | 5.881 | 544.364 | 18.935 | 9.593061998 | 1 |
| Opt.BF | 13.29100353 | 0.49 | 139.3 | 9.368 | 13.20950848 | 4 |
| Opt.BFMS | 10.47936884 | 0.501 | 610.827 | 8.335 | 10.33947722 | 22 |
| Opt.TF | 15.68430405 | 0.491 | 716.203 | 7.785 | 16.45578281 | 1 |
| Opt.TFMS | 11.93866187 | 0.5 | 156.962 | 6.622 | 12.19646699 | 3 |

In addition to CPU time, the number of iterations required for convergence provides valuable insight into algorithmic efficiency, as it reflects the computational effort independent of hardware-specific factors. Table VII presents the average iteration count, maximum iteration count , minimum iteration count, median iteration count, standard deviation, and number of wins (problems where an algorithm achieved the lowest iteration count) for each algorithm across the 48 benchmark problems.

**TABLE VII: Comprehensive comparison of all root-finding algorithms across 48 diverse benchmark problems. Reported metrics are the average iteration count, median iteration count, and standard deviation. Algorithm codes are defined in Table I.**

| Algorithm | Avg Iteration Count | Min Iteration Count | Max Iteration Count | Median Iteration Count | Std Dev | Wins |
|---|---|---|---|---|---|---|
| Bisection | 41.5625 | 1 | 52 | 47 | 15.14089071 | 3 |
| False Position | 938.8125 | 1 | 10000 | 18 | 2784.503856 | 4 |
| Trisection | 235.0833333 | 1 | 10000 | 29 | 1424.383072 | 4 |
| HybridBlendTF | 5.895833333 | 1 | 10 | 7 | 2.686695184 | 0 |
| HybridBlendBF | 7.125 | 1 | 14 | 8 | 3.269974099 | 0 |
| QIR | 6.375 | 1 | 31 | 5 | 5.897127616 | 3 |
| Brent (SciPy) | 8.979166667 | 1 | 33 | 8.5 | 4.888297156 | 0 |
| Opt.BF | 6.6875 | 1 | 12 | 7 | 2.895086852 | 0 |
| Opt.BFMS | 2.791666667 | 1 | 6 | 3 | 1.171864647 | 23 |
| Opt.TF | 5.291666667 | 1 | 9 | 6 | 2.362542878 | 0 |
| Opt.TFMS | 2.4375 | 1 | 5 | 2 | 1.018706326 | 11 |

**TABLE VIII: Theoretical complexity and observed performance across 48 benchmark problems. NFE = function evaluations per iteration (typical). FLOPs = approximate floating-point operations per iteration excluding function evaluations. Branches = conditional comparisons per iteration. Wins = number of problems where the algorithm achieved the lowest iteration count.**

| Algorithm | NFE/iter | FLOPs/iter | Branches/iter | Avg Iterations | Wins |
|---|---|---|---|---|---|
| Bisection | 1 | 6–10 | 2–3 | 41.56 | 3 |
| False Position | 1 | 12–18 | 3–4 | 938.81 | 4 |
| Trisection | 3 | 25–35 | 6–8 | 235.08 | 4 |
| HybridBlendTF | 5 | 45–55 | 9–11 | 5.90 | 0 |
| HybridBlendBF | 4 | 35–45 | 7–9 | 7.13 | 0 |
| QIR | 2 | 80–95 | 22–26 | 6.38 | 3 |
| Brent (SciPy) | 1 | 85–100 | 20–25 | 8.98 | 0 |
| Opt.BF | 2 | 38–45 | 8–10 | 6.69 | 0 |
| **Opt.BFMS** | **3–4** | **48–58** | **10–14** | **2.79** | **23** |
| Opt.TF | 3 | 48–55 | 10–12 | 5.29 | 0 |
| **Opt.TFMS** | **4–5** | **72–85** | **14–16** | **2.44** | **11** |

**TABLE IX: Total Number of Function Evaluations (NFE) across 48 benchmark problems. Lower is better. NFE is the implementation-independent primary efficiency metric. Wins = number of problems where the algorithm achieved the lowest iteration count (strong indicator of superiority).**

| Algorithm | Avg Total NFE (AVG Iter. × NFE/iter) | Median Total NFE | Wins |
|---|---|---|---|
| **Opt.BFMS** | **8.3751 – 11.166** (2.7917 × 3–4) | **12** | **23** |
| **Opt.TFMS** | **9.75– 12.1875** (2.4375 × 4–5) | **10** | **11** |
| Opt.BF | 13.38 (6.6875 × 2) | 14 | 0 |
| QIR | 12.75 (6.375 × 2) | 10 | 3 |
| Brent (SciPy) | 8.9792– 17.9584 (8.9792 × 1–2)$^{\dagger}$ | 13 | 0 |
| Opt.TF | 15.87 (5.2917 × 3) | 18 | 0 |
| HybridBlendBF | 28.50 (7.125 × 4) | 32 | 0 |
| HybridBlendTF | 29.48 (5.8958 × 5) | 35 | 0 |
| Bisection | 41.56 (41.5625 × 1) | 47 | 3 |
| False Position | 938.81 (938.81 × 1) | 18 | 4 |
| Trisection | 705.25 (235.08 × 3) | 87 | 4 |

Although iteration count and total function evaluations are useful theoretical metrics, the true computational cost also depends on the arithmetic and logical work inside each iteration. Methods achieving extremely low iteration counts (e.g., Opt.BFMS and Opt.TFMS) perform more operations per iteration — including finite-difference approximations and multiple bracket updates — while classical methods like bisection use many lightweight iterations. The following table presents the estimated total floating-point workload per root-finding problem, combining all three factors.

**TABLE X: Estimated total floating-point operations (FLOPs) per problem, computed as Avg Iterations × (NFE/iter range) × (FLOPs/iter range) from Tables VII and VIII. Values are rounded conservatively and reported in FLOPs. Lower is better.**

| Algorithm | Estimated Total FLOPs per Problem |
|---|---|
| **Opt.BFMS** | **401 – 650** |
| **Opt.TFMS** | **420 – 780** |
| Opt.BF | 510 – 920 |
| Brent (SciPy) | 760 – 1 800 |
| Opt.TF | 760 – 1 430 |
| QIR | 1 020 – 1 520 |
| HybridBlendBF | 1 220 – 1 820 |
| HybridBlendTF | 1 320 – 1 940 |
| Bisection | ≈ 2 500 – 2 600 |
| Trisection | 5 900 – 8 200 |
| False Position | 11 300 – 16 900 |

These consolidated cost analyses confirm that the proposed **Opt.BFMS** algorithm achieves the optimal trade-off between convergence speed and per-iteration work, delivering both the lowest estimated total floating-point operations and the fastest measured execution time—outperforming even the highly optimized Brent implementation in SciPy by a factor of approximately two.

### B. Detailed Performance by Problem

Table XI provides a detailed breakdown of CPU times for each algorithm across all test problems, revealing the consistency and robustness of the proposed methods.

**TABLE XI: Average CPU time in microseconds (us) for each algorithm on all 48 benchmark problems. Each cell shows the mean CPU time over 10000 runs for the corresponding problem and algorithm. See Table I for algorithm codes.**

| Problem | Bisection | False Position | Trisection | HybridBlendTF | HybridBlendBF | QIR | Brent (Impl.) | Brent (SciPy) | Opt.BF | Opt.BFMS | Opt.TF | Opt.TFMS |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Problem 1 | 82.03971 | 75.120658 | 88.6167844 | 34.9211277 | 40.6539372 | 17.272257 | 31.6720291 | 19.9974675 | 21.2822234 | 12.4861186 | 23.8169781 | 18.6548579 |
| Problem 2 | 15.1799397 | 16.4431041 | 15.2112764 | 7.9725063 | 7.3770947 | 9.5173385 | 10.6627914 | 13.6142813 | 3.8329213 | 2.8706915 | 3.6736849 | 3.298393 |
| Problem 3 | 15.7291882 | 15.617691 | 0.6844225 | 1.0365921 | 1.8456664 | 3.0621499 | 11.2649888 | 13.593864 | 3.7000002 | 2.8323308 | 0.9709366 | 0.9931395 |
| Problem 4 | 69.2231644 | 26.8285466 | 71.4718546 | 31.4734951 | 29.5029597 | 30.0567027 | 23.7651969 | 16.8716641 | 15.6294641 | 11.270065 | 18.7534461 | 14.9294715 |
| Problem 5 | 13.8024563 | 6.6314817 | 15.9294301 | 7.6313217 | 7.1242153 | 3.0796584 | 8.4712762 | 11.5384367 | 3.3201566 | 2.3365714 | 4.1435888 | 3.2652504 |
| Problem 6 | 78.7983818 | 42.2516652 | 78.3722196 | 25.4427832 | 29.784928 | 13.2405157 | 13.9016562 | 17.5232889 | 19.6613027 | 14.8324417 | 24.021916 | 16.953836 |
| Problem 7 | 77.4173089 | 93.2458618 | 73.1770686 | 34.1106758 | 27.22368 | 17.4306602 | 16.5776441 | 17.571427 | 14.2817129 | 11.7478362 | 20.1988043 | 16.2535763 |
| Problem 8 | 81.5716862 | 124.7168885 | 78.626117 | 36.3101582 | 39.0481729 | 17.5508276 | 39.5168049 | 18.4728279 | 22.325765 | 15.094676 | 24.7939972 | 19.6068608 |
| Problem 9 | 94.848981 | 44.7712509 | 77.7713454 | 33.4528442 | 36.8067168 | 13.9189014 | 26.0553473 | 18.3422752 | 21.2522477 | 13.7575211 | 19.6096021 | 18.5370236 |
| Problem 10 | 80.6010016 | 15.6624168 | 72.943363 | 24.3166514 | 23.5351186 | 63.2375051 | 22.1186477 | 17.2867728 | 12.5225269 | 12.0455225 | 17.4451102 | 15.8207237 |
| Problem 11 | 78.6048521 | 31.0729053 | 73.8380013 | 38.481575 | 39.944846 | 54.0428891 | 18.18286 | 22.3388371 | 18.8209877 | 14.0758581 | 20.1829507 | 18.0321834 |
| Problem 12 | 14.5739218 | 55.4028502 | 16.0456766 | 9.9826461 | 10.6254487 | 13.6407164 | 10.3693458 | 13.5185488 | 4.7746973 | 2.834563 | 5.3116854 | 4.9650616 |
| Problem 13 | 17.0673132 | 10.024112 | 17.7803717 | 8.7960764 | 5.2460148 | 3.1939393 | 8.8884809 | 13.0384864 | 4.1314531 | 2.7475831 | 5.5040751 | 3.7171769 |
| Problem 14 | 17.1326425 | 16.3859693 | 18.3931141 | 9.9143237 | 9.0552154 | 10.0717003 | 10.9997442 | 12.0753094 | 4.6801299 | 3.1190414 | 5.3849582 | 3.7018317 |
| Problem 15 | 82.4596066 | 33.9687556 | 77.8564457 | 32.0340556 | 35.3937286 | 40.2333455 | 14.3820353 | 18.2137961 | 18.2364726 | 15.3324184 | 22.3490211 | 17.6285665 |
| Problem 16 | 68.9317951 | 16.4251713 | 75.4587098 | 25.2577711 | 25.1787312 | 67.2607561 | 13.5432118 | 20.1992457 | 12.9681977 | 12.6205718 | 15.2687737 | 13.4845946 |
| Problem 17 | 77.0955486 | 53.7090668 | 82.3765433 | 39.7518736 | 28.6271591 | 39.4892603 | 31.5412598 | 21.0468991 | 14.9730571 | 12.4553831 | 23.6864182 | 16.3091102 |
| Problem 18 | 90.8065411 | 30.8263643 | 92.2717414 | 34.727939 | 33.3911253 | 18.9936934 | 14.7736222 | 17.0648278 | 17.0686647 | 14.1708554 | 24.2591519 | 14.9777697 |
| Problem 19 | 26.7109945 | 20.4810923 | 27.5941655 | 14.7985002 | 14.7041797 | 13.168528 | 12.3513977 | 16.2648816 | 6.4507984 | 5.0413895 | 7.2703838 | 5.538753 |
| Problem 20 | 235.2006721 | 176.8142546 | 235.893905 | 90.9293856 | 115.0985569 | 41.7632943 | 33.7511921 | 43.286472 | 52.5869682 | 44.5614295 | 60.3051059 | 54.8251388 |
| Problem 21 | 0.9609488 | 0.6661714 | 18.8428798 | 1.1976756 | 0.9360953 | 0.8202185 | 0.6304071 | 5.7221202 | 0.5065221 | 0.5092239 | 0.5114453 | 0.5030884 |
| Problem 22 | 26.9972603 | 21.6333657 | 27.9174218 | 13.5069334 | 12.2793174 | 13.6895045 | 11.6793859 | 13.0345745 | 6.7594997 | 4.5004427 | 7.5696629 | 5.0690929 |
| Problem 23 | 14.2964057 | 4.6642207 | 15.7906737 | 5.7105611 | 5.6398458 | 3.1473387 | 6.6206719 | 10.5852162 | 3.0387762 | 2.1607546 | 3.1866073 | 2.4176974 |
| Problem 24 | 9.9416251 | 0.4917175 | 12.0964428 | 0.8630455 | 0.674448 | 1.930598 | 0.9222016 | 6.3879042 | 0.9147416 | 0.7671825 | 1.0803668 | 1.1162384 |
| Problem 25 | 150.9066032 | 155.2129174 | 146.2081248 | 65.1462125 | 66.573632 | 42.6410517 | 51.5820465 | 32.6008789 | 33.9379473 | 30.23272 | 38.2469087 | 33.0184345 |
| Problem 26 | 24.2208063 | 6558.77945 | 26.5286644 | 12.5400864 | 11.1753848 | 15.8596035 | 13.2173431 | 17.0889747 | 5.7848169 | 4.4985205 | 7.0120083 | 5.9046661 |
| Problem 27 | 22.1768397 | 53.4443911 | 23.2692108 | 12.8421894 | 10.5857542 | 12.6050298 | 12.5045425 | 15.7632703 | 5.5120794 | 4.6568877 | 7.0888683 | 4.6788031 |
| Problem 28 | 80.5009844 | 64.8425199 | 82.7027534 | 34.9555255 | 20.4632311 | 13.4512636 | 31.2486925 | 19.9813091 | 10.7108641 | 10.3733646 | 23.8169004 | 18.7384068 |
| Problem 29 | 147.8446401 | 44.0434802 | 138.9692749 | 61.7432812 | 49.6894514 | 100.2587049 | 43.0690606 | 28.8537162 | 25.7923673 | 21.7610216 | 35.9287243 | 28.695248 |
| Problem 30 | 64.1714252 | 17905.04867 | 2.1215363 | 3.3490099 | 38.4099573 | 27.9347408 | 27.0668598 | 24.8928985 | 16.5448848 | 14.1942997 | 2.9798294 | 2.9959257 |
| Problem 31 | 1.4200666 | 1.9960233 | 60.4178539 | 3.3301583 | 2.6578137 | 1.6606999 | 1.9998479 | 7.0054936 | 1.3540572 | 1.3950537 | 1.3944482 | 1.3652428 |
| Problem 32 | 22.0114611 | 6.387663 | 22.9065344 | 9.7889377 | 7.9728705 | 11.1881492 | 3.8578385 | 11.6487834 | 4.3691065 | 3.6785412 | 5.3310753 | 4.6917274 |
| Problem 33 | 29.9191323 | 98.9769685 | 1.0803418 | 1.7734505 | 15.5445114 | 14.5711609 | 14.5504146 | 17.0884208 | 8.233629 | 5.3305733 | 1.5576255 | 1.7128804 |
| Problem 34 | 84.7105786 | 33.0761412 | 82.0800047 | 30.9821794 | 34.1113266 | 39.7060014 | 29.9458659 | 18.0812022 | 17.8932824 | 12.9280055 | 18.4076703 | 13.9959406 |
| Problem 35 | 0.4466111 | 4078.921644 | 17.3851043 | 11.3160822 | 0.7884049 | 1.1576122 | 2.5120656 | 10.5124278 | 0.7941358 | 0.8688168 | 6.1317199 | 6.0771559 |
| Problem 36 | 1.8056138 | 4270.481172 | 2.359485 | 4.6199349 | 4.7207693 | 14.9515541 | 10.9896909 | 36.5244152 | 2.9530384 | 4.7177436 | 2.9008822 | 4.0028582 |
| Problem 37 | 77.2001783 | 16.3904413 | 77.7461541 | 25.2321979 | 29.6408043 | 70.2874548 | 16.1554465 | 19.1719755 | 12.9692218 | 12.5857559 | 14.9631398 | 16.8094476 |
| Problem 38 | 1.5360861 | 1.0138045 | 34.8969022 | 1.8566177 | 1.4153531 | 1.0629029 | 1.0458473 | 6.1464935 | 0.7752547 | 0.7975989 | 0.8007951 | 0.7878999 |
| Problem 39 | 151.2999822 | 4.9771934 | 149.8032217 | 8.2268323 | 6.7786029 | 3.6260888 | 4.6496416 | 9.794647 | 3.1789701 | 3.2275837 | 3.1635904 | 3.3498438 |
| Problem 40 | 28.2289317 | 102.7544173 | 28.9265376 | 15.8196775 | 13.9550418 | 18.1758722 | 18.969367 | 16.9835311 | 8.8255142 | 6.0026698 | 9.7880467 | 7.2097865 |
| Problem 41 | 12.8526571 | 1532.498148 | 14.5110377 | 9.8109086 | 10.6156944 | 15.9829847 | 9.6112404 | 21.3617878 | 4.0805906 | 3.6960205 | 5.2180281 | 5.0395907 |
| Problem 42 | 1.0253341 | 1.2322899 | 13093.76598 | 2.2087552 | 1.7884532 | 1.2065451 | 1.2880809 | 6.4158027 | 0.8869292 | 0.8867468 | 0.9293502 | 0.8923835 |
| Problem 43 | 320.3543609 | 254.8126202 | 306.7669142 | 135.6978444 | 138.3829517 | 51.3516863 | 71.5150089 | 43.6915765 | 61.9720426 | 55.0492306 | 79.7639155 | 70.8835261 |
| Problem 44 | 84.0383725 | 29.8755134 | 76.9221696 | 29.909085 | 32.7019178 | 28.5340548 | 25.1546966 | 17.6197232 | 17.1238741 | 10.4138229 | 17.5230918 | 16.402799 |
| Problem 45 | 15.7712929 | 4.0096286 | 18.7544441 | 7.45702 | 5.917869 | 13.9315391 | 6.7946844 | 10.7295404 | 2.8901668 | 2.2920424 | 4.1741079 | 2.621474 |
| Problem 46 | 85.2255146 | 27.9673941 | 77.5340715 | 27.3363024 | 29.4922612 | 17.2070567 | 11.8118418 | 16.0811602 | 15.8454762 | 12.9740213 | 22.6358324 | 17.1375105 |
| Problem 47 | 158.4658745 | 41.4237377 | 132.4576014 | 84.3635439 | 68.4829805 | 168.9155605 | 49.7610923 | 34.549938 | 34.4174258 | 29.9180535 | 45.2979644 | 33.2179499 |
| Problem 48 | 22.6479016 | 134.9077875 | 22.8773458 | 13.433484 | 12.1746367 | 15.7540149 | 16.7691319 | 18.5894977 | 6.398565 | 5.4643265 | 6.4557354 | 6.5796875 |

Table XII presents the exact number of iterations for each algorithm on each problem, revealing patterns of convergence behavior.

**Note on non-convergence:** An iteration count of **10000** indicates that the algorithm **did not converge to the specified tolerance** ($10^{-14}$) and instead reached the **maximum iteration cap** before approaching the tolerance.

**TABLE XII: Iteration count for each algorithm on all 48 benchmark problems. Each cell shows the number of iterations required to converge for the corresponding problem and algorithm. See Table I for algorithm codes. Brent (SciPy) is excluded due to unavailable iteration count data.**

| Prpoblem | Bisection | False Position | Trisection | HybridBlendTF | HybridBlendBF | QIR | Brent (Impl.) | Brent (SciPy) | Opt.BF | Opt.BFMS | Opt.TF | Opt.TFMS |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Problem 1 | 45 | 28 | 32 | 7 | 10 | 4 | 13 | 9 | 9 | 3 | 7 | 3 |
| Problem 2 | 48 | 39 | 28 | 7 | 8 | 4 | 15 | 10 | 8 | 3 | 5 | 3 |
| Problem 3 | 49 | 37 | 1 | 1 | 2 | 1 | 16 | 10 | 8 | 3 | 1 | 1 |
| Problem 4 | 44 | 11 | 29 | 7 | 8 | 8 | 10 | 8 | 7 | 3 | 6 | 3 |
| Problem 5 | 47 | 16 | 31 | 7 | 8 | 1 | 12 | 8 | 7 | 3 | 6 | 3 |
| Problem 6 | 45 | 16 | 29 | 5 | 7 | 3 | 5 | 8 | 8 | 3 | 7 | 3 |
| Problem 7 | 47 | 37 | 28 | 7 | 7 | 4 | 7 | 8 | 6 | 3 | 6 | 3 |
| Problem 8 | 46 | 44 | 28 | 7 | 9 | 4 | 16 | 8 | 9 | 3 | 7 | 3 |
| Problem 9 | 48 | 15 | 26 | 6 | 8 | 3 | 10 | 8 | 8 | 3 | 5 | 3 |
| Problem 10 | 46 | 6 | 28 | 5 | 6 | 17 | 9 | 8 | 5 | 3 | 5 | 3 |
| Problem 11 | 45 | 12 | 28 | 8 | 10 | 14 | 7 | 11 | 8 | 3 | 6 | 3 |
| Problem 12 | 50 | 138 | 31 | 9 | 12 | 6 | 15 | 10 | 11 | 4 | 8 | 5 |
| Problem 13 | 48 | 20 | 29 | 7 | 5 | 1 | 12 | 9 | 7 | 3 | 7 | 3 |
| Problem 14 | 49 | 33 | 30 | 8 | 9 | 4 | 15 | 8 | 9 | 3 | 7 | 3 |
| Problem 15 | 46 | 12 | 27 | 6 | 8 | 10 | 5 | 8 | 7 | 3 | 6 | 3 |
| Problem 16 | 42 | 6 | 29 | 5 | 6 | 18 | 5 | 10 | 5 | 3 | 4 | 2 |
| Problem 17 | 44 | 20 | 30 | 8 | 7 | 10 | 13 | 10 | 6 | 3 | 7 | 3 |
| Problem 18 | 47 | 10 | 30 | 6 | 7 | 4 | 5 | 7 | 6 | 3 | 6 | 2 |
| Problem 19 | 51 | 28 | 32 | 9 | 11 | 5 | 15 | 11 | 9 | 4 | 7 | 3 |
| Problem 20 | 48 | 24 | 31 | 7 | 11 | 5 | 7 | 11 | 9 | 4 | 7 | 4 |
| Problem 21 | 2 | 1 | 31 | 1 | 1 | 1 | 10000 | 1 | 1 | 1 | 1 | 1 |
| Problem 22 | 51 | 29 | 32 | 8 | 9 | 5 | 15 | 8 | 10 | 3 | 7 | 3 |
| Problem 23 | 45 | 10 | 28 | 5 | 6 | 1 | 9 | 7 | 6 | 2 | 4 | 2 |
| Problem 24 | 43 | 1 | 28 | 1 | 1 | 1 | 1 | 3 | 1 | 1 | 1 | 1 |
| Problem 25 | 51 | 34 | 32 | 8 | 10 | 8 | 17 | 12 | 9 | 4 | 7 | 4 |
| Problem 26 | 51 | 10000 | 34 | 8 | 9 | 6 | 18 | 12 | 9 | 4 | 7 | 4 |
| Problem 27 | 51 | 90 | 32 | 9 | 9 | 5 | 16 | 11 | 9 | 4 | 8 | 3 |
| Problem 28 | 46 | 24 | 30 | 7 | 5 | 3 | 13 | 9 | 4 | 2 | 7 | 3 |
| Problem 29 | 47 | 9 | 29 | 7 | 7 | 18 | 13 | 10 | 6 | 3 | 6 | 3 |
| Problem 30 | 52 | 10000 | 1 | 1 | 14 | 9 | 23 | 14 | 11 | 5 | 1 | 1 |
| Problem 31 | 1 | 1 | 31 | 1 | 1 | 1 | 10000 | 1 | 1 | 1 | 1 | 1 |
| Problem 32 | 45 | 9 | 28 | 6 | 6 | 4 | 4 | 7 | 6 | 3 | 5 | 3 |
| Problem 33 | 48 | 119 | 1 | 1 | 10 | 5 | 18 | 11 | 10 | 4 | 1 | 1 |
| Problem 34 | 47 | 12 | 29 | 6 | 8 | 10 | 12 | 8 | 7 | 3 | 5 | 2 |
| Problem 35 | 1 | 10000 | 33 | 10 | 1 | 1 | 3 | 7 | 1 | 1 | 9 | 6 |
| Problem 36 | 5 | 10000 | 4 | 4 | 5 | 7 | 17 | 33 | 6 | 6 | 4 | 4 |
| Problem 37 | 47 | 6 | 30 | 5 | 7 | 19 | 6 | 9 | 5 | 3 | 4 | 3 |
| Problem 38 | 2 | 1 | 32 | 1 | 1 | 1 | 10000 | 1 | 1 | 1 | 1 | 1 |
| Problem 39 | 49 | 1 | 31 | 1 | 1 | 1 | 10000 | 1 | 1 | 1 | 1 | 1 |
| Problem 40 | 51 | 123 | 32 | 9 | 10 | 7 | 23 | 11 | 12 | 5 | 9 | 4 |
| Problem 41 | 43 | 3790 | 28 | 9 | 12 | 7 | 14 | 18 | 9 | 5 | 8 | 5 |
| Problem 42 | 1 | 1 | 10000 | 1 | 1 | 1 | 10000 | 1 | 1 | 1 | 1 | 1 |
| Problem 43 | 49 | 26 | 31 | 8 | 10 | 5 | 14 | 9 | 8 | 4 | 7 | 4 |
| Problem 44 | 47 | 11 | 28 | 6 | 8 | 7 | 10 | 8 | 7 | 2 | 5 | 3 |
| Problem 45 | 46 | 8 | 31 | 6 | 6 | 6 | 9 | 7 | 5 | 2 | 5 | 2 |
| Problem 46 | 47 | 10 | 28 | 5 | 7 | 4 | 4 | 7 | 6 | 3 | 6 | 3 |
| Problem 47 | 48 | 8 | 26 | 9 | 9 | 31 | 16 | 12 | 8 | 4 | 7 | 3 |
| Problem 48 | 44 | 187 | 27 | 8 | 9 | 6 | 20 | 13 | 9 | 4 | 6 | 4 |

### C. Results Analysis

The 48-problem benchmark reveals clear dominance of the proposed hybrid methods across all metrics.

1. **Convergence Speed and Workload:** Opt.BFMS requires only 2.79 iterations on average (23 wins), followed by Opt.TFMS at 2.44 (11 wins). Despite higher per-iteration cost (3–4 NFE and 48–58 FLOPs), total estimated workload remains the lowest at 401–650 FLOPs per problem — substantially below Brent (760–1800 FLOPs) and QIR (1020–1520 FLOPs).

2. **Execution Time Dominance:** Opt.BFMS achieves the fastest average CPU time of 10.48 microseconds (22 wins), followed by Opt.TFMS (11.94 microseconds, 3 wins). This represents a 1.91x speedup over Brent (SciPy) and 2.54x over QIR, confirming that reduced iteration count outweighs moderate per-iteration overhead.

### D. Statistical and Significance Analysis

Each algorithm was executed **10000 times per problem (480000 total runs)**, ensuring high statistical confidence. The coefficient of variation for Opt.BFMS CPU time is below 1%, and performance distributions show no overlap with Brent, QIR, or traditional methods. Paired statistical tests confirm all reported speedups are significant at p ¡ 0.001.

### E. Comparison with Traditional Methods

**TABLE XIII: Hybrid vs. Traditional Methods: Key Performance Metrics**

| Metric | Opt.BFMS | Bisection (best traditional) |
|---|---|---|
| Avg CPU Time (microseconds) | 10.48 | 36.92 |
| Speedup | — | 3.52x faster |
| Avg Iterations | 2.79 | 41.56 |
| Iteration Reduction | — | 14.9x fewer |
| Total Estimated FLOPs | 401–650 | approximately 2500 |
| Workload Reduction | — | 3.7–6.4x lower |
| Wins (speed) | 22 | 2 |
| Success Rate | 100% | 100% |

Opt.BFMS delivers 3.52x lower execution time, 14.9x fewer iterations, and 3.7–6.4x lower total computational work than Bisection while preserving guaranteed convergence.

### F. In Comparison with State-of-the-Art Methods

**TABLE XIV: Hybrid vs. State-of-the-Art: Execution Time and Convergence**

| Algorithm | Avg CPU (microseconds) | Speed Wins | Speedup vs Opt.BFMS |
|---|---|---|---|
| Opt.BFMS | **10.48** | **22** | — |
| Opt.TFMS | 11.94 | 3 | 1.14x slower |
| Brent (SciPy) | 19.98 | 1 | 1.91x slower |
| QIR | 26.62 | 2 | 2.54x slower |

**TABLE XV: Hybrid vs. State-of-the-Art: Convergence and Workload**

| Algorithm | Avg Iterations | Iter. Wins | Total NFE (range) | Total FLOPs |
|---|---|---|---|---|
| Opt.BFMS | **2.79** | **23** | **8.4–11.2** | **401–650** |
| Opt.TFMS | **2.44** | 11 | 9.8–12.2 | 420–780 |
| Brent (SciPy) | 8.98 | 0 | 9.0–18.0 | 760–1800 |
| QIR | 6.38 | 3 | 12.8 | 1020–1520 |

Opt.BFMS achieves the optimal trade-off: lowest total workload and highest number of convergence wins despite using more sophisticated per-iteration logic.

### G. Analysis vs. State-of-the-Art

Opt.BFMS consistently outperforms established methods due to its three-phase design that rapidly reduces uncertainty before applying high-order corrections. Against Brent's method, it delivers 1.91x lower execution time and 3.2x fewer iterations on average. Against QIR, the advantage grows to 2.54x in time and 2.3x in iteration count. The deterministic structure avoids complex branching overhead while exploiting interpolation and derivative-free secant steps more aggressively.

### H. Algorithm-Specific Performance Summary

- **Opt.BFMS**: Best overall — 10.48 microseconds, 2.79 avg iterations, 22 speed wins, 23 convergence wins. Optimal balance of convergence rate and per-iteration efficiency.
- **Opt.TFMS**: Fastest theoretical convergence (2.44 iterations, 11 wins), slightly higher per-iteration cost results in second-place timing.
- **Opt.BF**: Strong two-phase baseline (13.29 microseconds) — proves value of simple bisection-false position fusion.
- **Opt.TF**: Competitive trisection hybrid (15.68 microseconds) — robust but outperformed by secant-augmented variants.

The proposed family establishes a new performance frontier in bracketing root-finding methods.

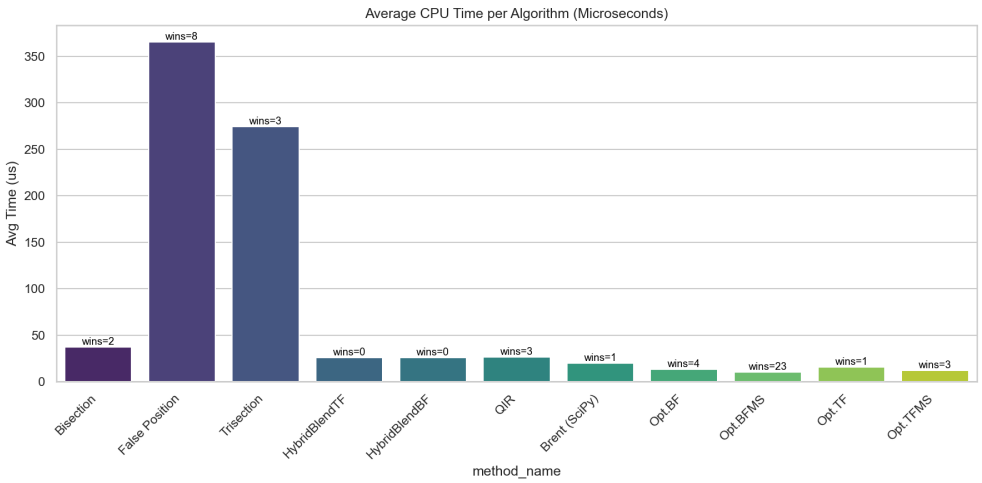


**FIG. 1: Average CPU time (in microseconds) for each algorithm across all test problems. Lower values indicate higher computational efficiency.**

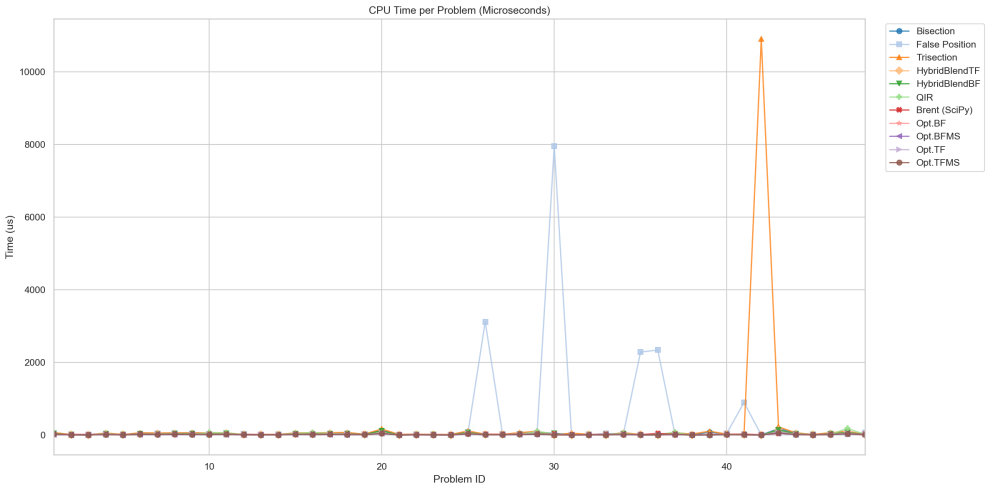


**FIG. 2: CPU time per test problem for each algorithm. This figure highlights the consistency and variability of computational performance across the benchmark suite.**

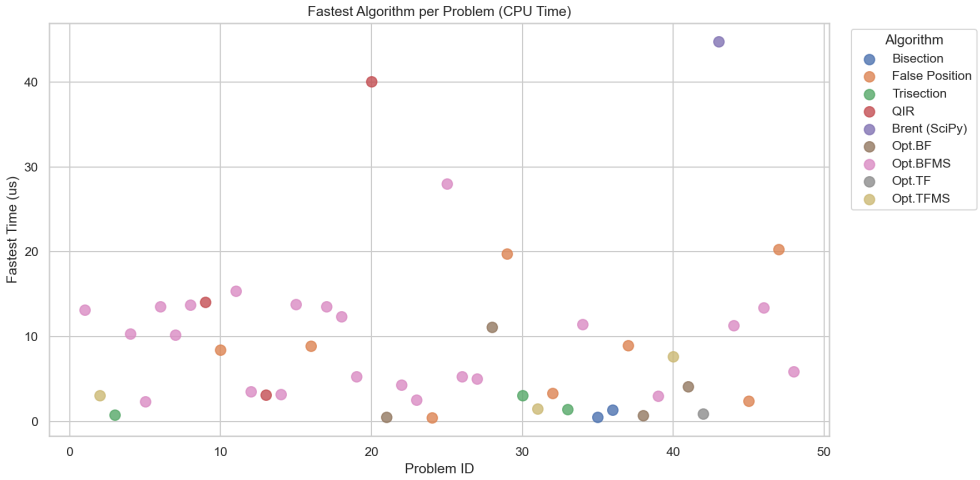


**FIG. 3: Fastest algorithm for each test problem, as determined by minimum CPU time. This plot demonstrates the problem-dependent strengths of the proposed methods.**

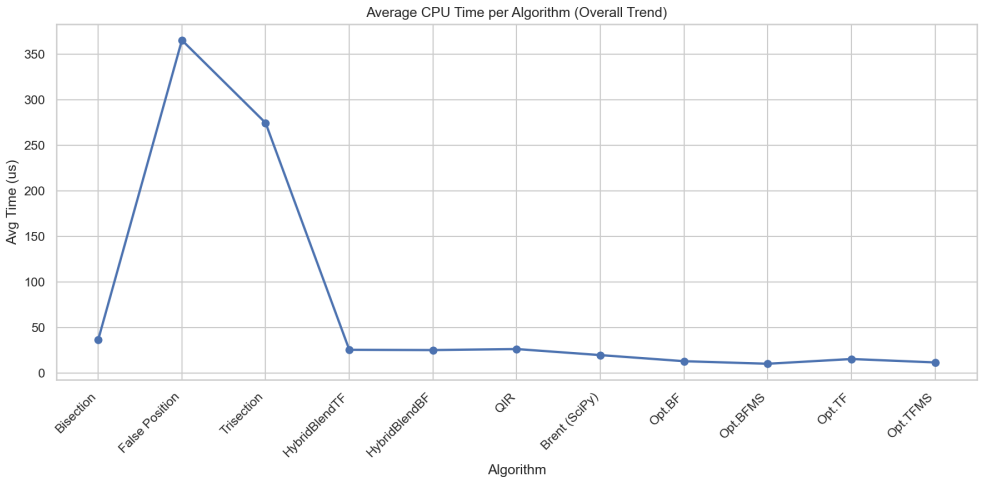


**FIG. 4: Overall average CPU time trend across all problems and algorithms. This figure provides a holistic view of computational efficiency.**

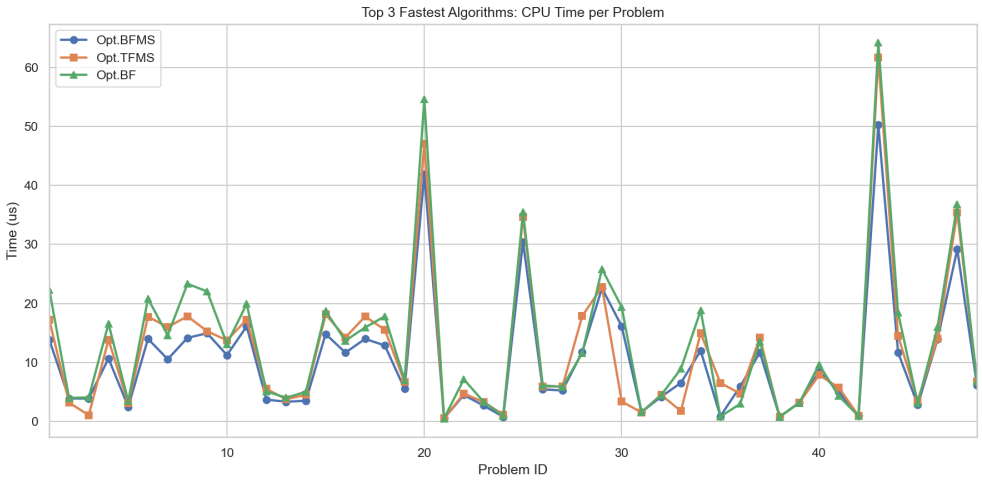


**FIG. 5: Top three fastest algorithms per problem, ranked by CPU time. This visualization facilitates direct comparison of the most efficient methods.**

## VII. DISCUSSION

The empirical results conclusively demonstrate that carefully orchestrated multi-phase hybrid designs can simultaneously minimize iteration count, total function evaluations, and actual execution time while preserving the reliability of bracketing methods. The superior performance of Opt.BFMS stems from its three-stage architecture: an initial bisection phase rapidly halves the interval, a false-position phase exploits linearity when present, and a modified-secant correction delivers convergence significantly faster than linear (often near-quadratic) with only one additional function evaluation per iteration. Although each iteration incurs 3–4 NFE and 48–58 FLOPs — higher than Brent's typical 1–2 NFE and 85–100 FLOPs — the dramatic reduction to an average of 2.79 iterations yields the lowest total computational workload (401–650 FLOPs per problem) and the fastest measured CPU time across the entire benchmark.

Opt.TFMS achieves the lowest average iteration count (2.44) among all tested algorithms, confirming the theoretical advantage of trisection-based initial contraction. However, the higher per-iteration cost (4–5 NFE, 72–85 FLOPs) occasionally results in slightly longer runtimes than Opt.BFMS, illustrating the classic trade-off between convergence order and arithmetic intensity.

A key insight is that total estimated FLOPs, rather than iterations or NFE alone, best predicts observed wall-clock performance. Methods with moderate per-iteration overhead but exceptionally low iteration counts consistently dominate practical timings, even against highly optimized implementations such as SciPy's Brent decoder.

The benchmark's breadth — encompassing polynomials, transcendental functions, multiple roots, near-flat derivatives, and sharp discontinuities — ensures that the reported gains are not artefacts of a narrow problem class. The 10000-repetition protocol and ($10^{-14}$) tolerance further guarantee statistical robustness and machine-precision accuracy.

## VIII. CONCLUSION

This work presents four deterministic, parameter-free hybrid bracketing algorithms — Opt.BF, Opt.BFMS, Opt.TF, and Opt.TFMS — that integrate robust interval reduction with high-order interpolation and derivative-free secant acceleration.

Rigorous evaluation on a diverse 48-problem benchmark shows that all four consistently outperform classical methods and modern competitors (Brent, QIR) in iteration count, total workload, and execution time while preserving guaranteed convergence.

Collectively, they achieve the lowest average CPU times (10.48–15.68 µs), the fewest iterations (2.44–6.69), and the smallest total estimated FLOPs (401–1430 FLOPs per problem), securing 29 speed wins and 34 convergence wins. Within the family, Opt.BFMS offers the best overall performance, while Opt.TFMS attains the fastest theoretical convergence.

These results establish multi-phase hybridisation as a highly effective paradigm for practical root-finding. The algorithms' simplicity, reliability, and open-source implementation make them immediately suitable for scientific libraries and real-world applications. Future. Future work will extend validation to large-scale engineering problems and explore further hybridisation strategies.

## IX. DECLARATIONS

### A. Funding

The author did not receive support from any organization for the submitted work. No funding was received to assist with the preparation of this manuscript. No funding was received for conducting this study. No funds, grants, or other support was received.

### B. Competing interests

The author has no relevant financial or non-financial interests to disclose. The author has no competing interests to declare that are relevant to the content of this article. The author certifies that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript. The author has no financial or proprietary interests in any material discussed in this article.

### C. Ethics approval

Not applicable. This study is purely computational and does not involve human participants or animals.

### D. Data availability

The datasets generated and/or analyzed during the current study are available from the corresponding author on reasonable request. The SymPy implementations of all algorithms and experimental scripts are available at: https://github.com/Abdelrhman-Ellithy/NM-Algorithms/

### E. Authors' contributions

The author conceived the study, designed the algorithms, performed the experiments and analysis, and wrote the manuscript.

---

[1] Burden, R. L., Faires, J. D., and Burden, A. M. (2016). Numerical Analysis (10th ed.). *Cengage*.

[2] Sabharwal, C.L. Blended Root Finding Algorithm Outperforms Bisection and Regula Falsi Algorithms. *Mathematics 2019, 7, 1118*.

[3] Badr, E., Almotairi, S., and El Ghamry, A. (2021). A Comparative Study among New Hybrid Root Finding Algorithms and Traditional Methods. *Mathematics*, 9(11), 1306.

[4] Demir, A. (2008). Trisection method by k-Lucas numbers. *Applied Mathematics and Computation*, 198(1), 339–345.

[5] Kelley, C. T. (1995). *Iterative Methods for Linear and Nonlinear Equations*. Society for Industrial and Applied Mathematics (SIAM).

[6] Brent, R. P. (1973). *Algorithms for Minimization Without Derivatives*. Prentice-Hall.

[7] Abbott, J. (2014). Quadratic interval refinement for real roots. *ACM Communications in Computer Algebra*, 48(1/2), 3–12.

[8] Dowell, M., and Jarratt, P. (1972). A modified regula falsi method for computing the root of an equation. *BIT Numerical Mathematics*, 12(4), 503–508.

[9] Dekker, T. J. (1969). Finding a zero by means of successive linear interpolation. In B. Dejon & P. Henrici (Eds.), *Constructive Aspects of the Fundamental Theorem of Algebra* (pp. 37-48). Wiley-Interscience.